ETH zürich

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

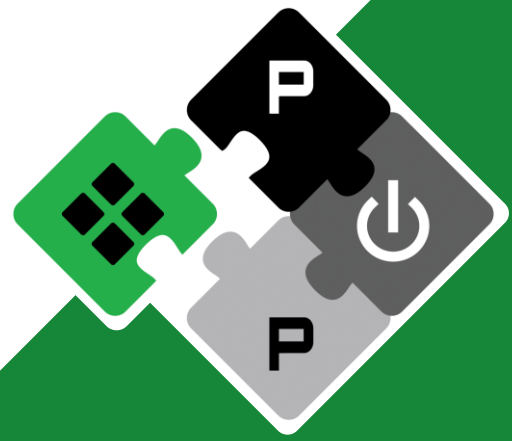
CachePool: Many-core cluster of customizable, lightweight scalar-vector PEs for irregular L2 data-plane workloads

Integrated Systems Laboratory (ETH Zürich)

Zexin Fu, Diyou Shen zexifu, dishen@iis.ee.ethz.ch

Alessandro Vanelli-Coralli avanelli@iis.ee.ethz.ch

Luca Benini lbenini@iis.ee.ethz.ch



@pulp_platform

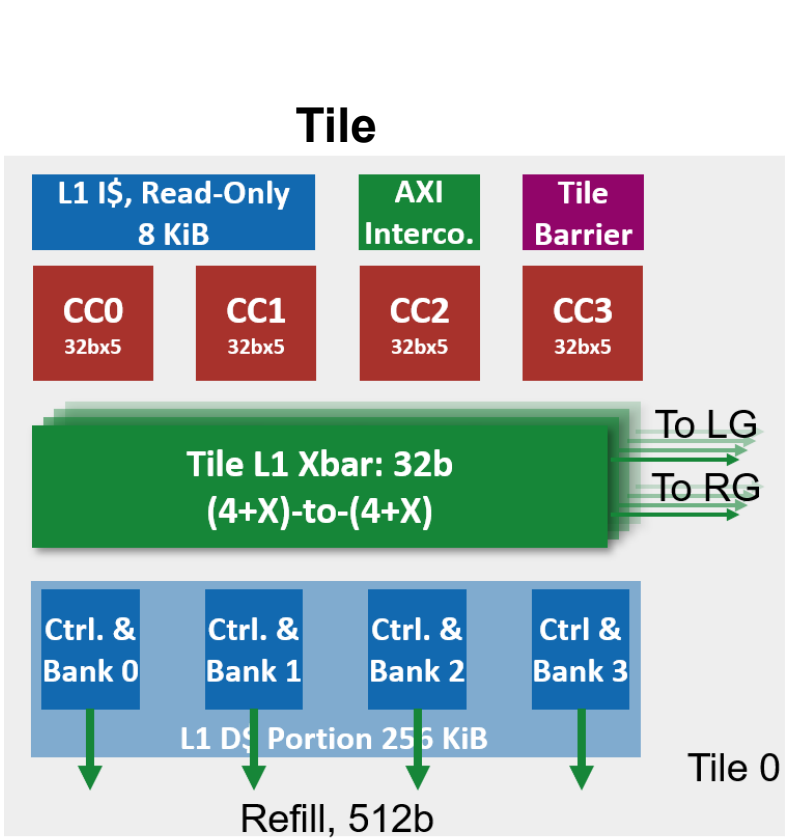
pulp-platform.org

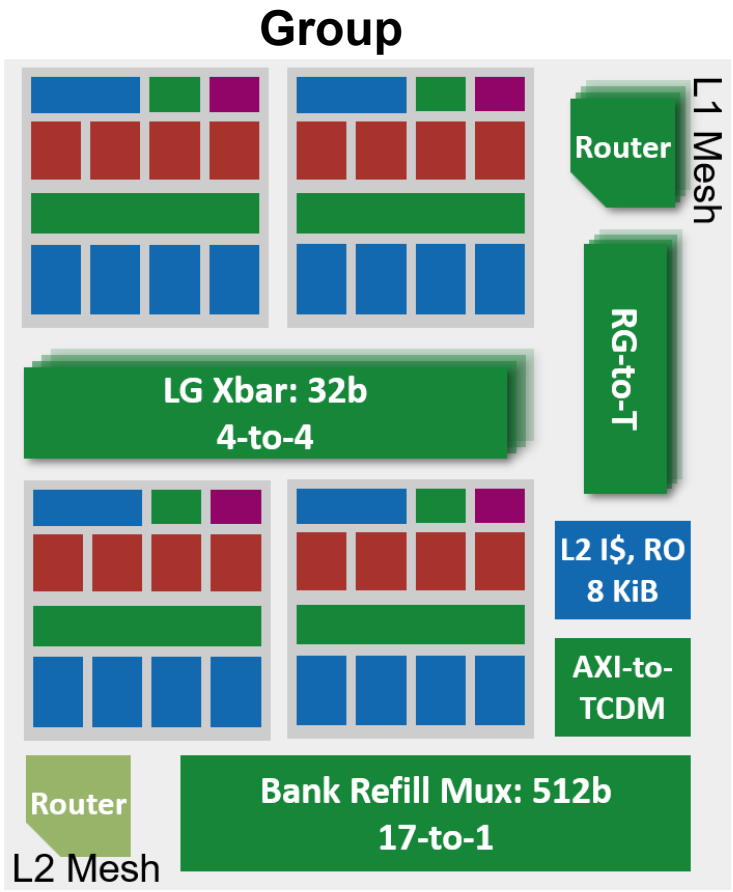
youtube.com/pulp_platform

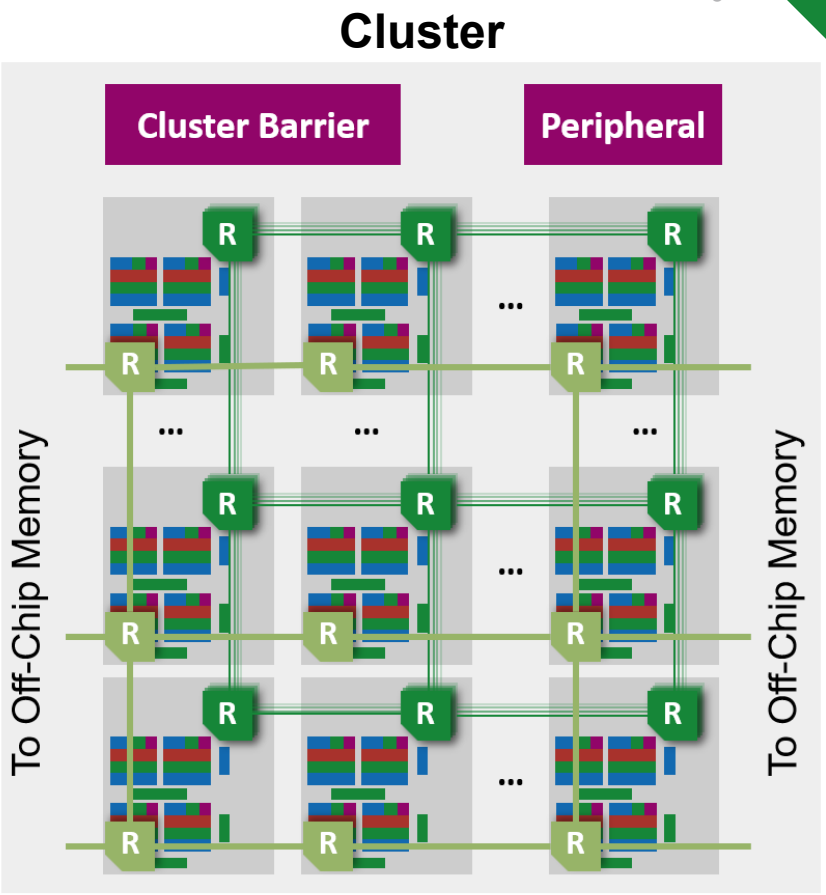
PULP Platform

Open Source Hardware, the way it should be!

Hardware Development (Cluster)







Hardware Development

- **Plans and Status**
  - Benchmarking and performance tuning
  - Some nice to have features
    - **Group-level partitioning (Merged)**
      Add by-group partitioning support. Switchable in software runtime.
    - **More L2 channel support (Merged)**
      Support both 8 and 16 channels for off-chip connection. (TeraPool uses 16 channels)
    - **Cache bypass data path (Final Cleanup)**
      Add a configurable bypass path. We can set the bypass address in software, and enable/disable it
      RTL implementation done, now doing final cleanups before merging, expected done in 2 days.
- **PR merged**
  - New RLC kernel, including some small RTL fixes and a short version in CI.
  - Two features above
  - A timing path cut between Snitch and Spatz (no performance degradation observed)

Report

- **Status**
  - Introduction (>95%)
    - Modified from annual report. May need some final tuning later
  - Architecture (>95%)
    - Folded cache added
    - Final edits according to your previous feedback
  - Physical Design (80%)
    - Latest results in. Some final tuning and details needed.
    - Dual-Snitch version PD done, not yet add results into text.
  - GVSoC (90%)
    - Basic structure completes. Need to add more details
  - Runtime (>95%)
    - In general completes. May need some final cleanups.

Report

- **Status**
  - Kernel Characterization (~30%)
    - Based on annual report version
    - Needs to adjust some parts after non-linear support integrated
  - RLC Implementation (~90%)
    - Based on annual report version
    - Need some major updates since kernel has been updated
  - Evaluation (50%)
    - GVSoC-related part complete
    - Working on RTL result collection. (GEMM, GEMV done; hot cache util >90% and >60% with all cores)
  - Appendix (50%)
    - Currently missing: NoC topology exploration (student is writing her final report)
    - Need to improve: WT-L0.5

Next Step

- **Final report writing**
  - We will try to finalize the first complete version of report this week and check with Luca.

- **Final codespace cleanup**
  - Will prepare a tag (v0.1.0) on github for delivery

Timeline

		2025							2026					
		11	12	1	2	3	4	5	6	7	8	9	10	11
WP1 ManyCore Architecture	Multi-Tile Scaling [RTL] Complete													
	4/16-Group Scaling [GVSoC + RTL] Complete													
	Interco. Optimization (multi-group) [RTL] Almost Complete													
	Timing optimization for NoC [RTL] Ongoing by colleague													
	NoC Topology Exploration [RTL, GVSoC] Ongoing by student													
WP2 Cache	Cache partitioning [RTL] Complete													
	WT-L1 by master student [RTL] Complete													
	Folded Cache [GVSoC + RTL] Complete													

Timeline

		2025		2026										
		11	12	1	2	3	4	5	6	7	8	9	10	11
WP3 VPE	Shared vector core [RTL + GVSoC] Almost Complete													
WP4 Benchmark	Single-user kernel optimization [SW] Ongoing													
	Multi-user and more scenarios [SW] Ongoing													
Report	Final Report and Benchmark													

L1 Access Latency

- **Problem: increased L1 access latency when scaling up**
- **Solutions**
  - **(Complete, RTL & Physical) Hardware optimization**
    Further optimize the interconnection and cache controller to reduce access latency on existing architecture
    => Retiming the current interconnection, remove unnecessary cuts in interco and cache
    => Target to reduce 1-2 cycles (~5 cyc) for hit-to-use latency
    Currently has reduced to 6 cycles:
    Core => Xbar => AMO => Ctrl (2) => AMO => Core
  - **(Complete, RTL) Cache partitioning**
    Partition the shared L1 to keep data localized to avoid remote access latency
  - **(Complete with some problems, RTL & Physical) WT-L1**
    Explore and evaluate a private WT L1 for each core
    => Performance degrade on single tile
    => Needs more work to support larger system

L1 Access Latency

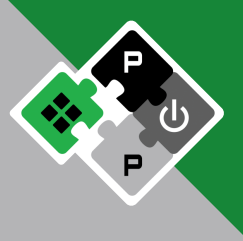
- **Problem: large NoC latency (~28 cyc based on previous exploration)**
- **Solutions**
  - **(Complete, Failed, Physical) Reduce hop latency**
    The long wire length requires us to have two cuts
  - **(Cancelled due to time, Physical) Cut clk-domain (optional)**
    If 1-cyc is not possible, explore to operate the NoC under a higher frequency than cores (e.g. 1.5GHz)
    => Effectively cut the latency by 1/3
  - **(Complete, GVSoC & Physical) NoC Topology**
    Torus topology cannot reduce average latency due to extra cuts needed for longer wires
    However, it can help improve the bandwidth, which can help improving the perfromance

PE-Related

- **Problem: Improve scalar core performance**
- **Solutions**
  - **(Complete, RTL & GVSoC) Shared vector core**
    Explore two Snitch sharing one Spatz configuration to improve scalar performance
    Software complexity needs to be considered, especially on the possible conflicting use of register file.

RLC Workload

- **Problem: Current RLC model cannot reveal all properties of the kernel**
- **Solutions**
  - **(Complete, SW) Kernel optimization**
    Adapt the current model to better represent the real cases, e.g. do some operations on the data. Further cooperation with HQ side to develop a more complete and better model.
  - **(Complete, SW) Kernel development**
    Implement the missing use cases for multi-user



Thank you!



